

National University of Computer and Emerging Sciences

Islamabad Campus | Department of Computer Science

CS-2001: Data Structures

Assignment 2: ImageLab (Phase 1 Technical Report)

Student Information

Student Name:	Ahmad Ali
Roll Number:	25-i0817
Section:	BS CS-E
Course:	CS-2001 Data Structures
Submission:	Phase 1 Deliverables (16/10/2026)

Project Abstract & Phase 1 Scope:
This report presents the complete architectural design, data structure representations, algorithmic complexities, and empirical test results for **Phase 1 of the ImageLab Studio**. ImageLab is implemented from foundational principles without the use of C++ Standard Template Library (STL) containers, `enum` constructs, or `inline` functions. Phase 1 provides:

1. Pure 2D pointer-based pixel matrices (`Pixel**`) for row-level manipulation and vertical reflection swapping.
2. Custom template bounded dynamic array Stack (`stackType<T>`) for transaction history.
3. Custom template circular ring buffer Queue (`queueType<T>`) for execution pipeline dispatch.
4. A dual-stack undo/redo engine and dual flood-fill algorithms (Stack DFS and Queue BFS).
5. Automated regression testing adhering strictly to all minimum criteria outlined in Section 6.8.

1 Data Structure Representations and Justifications

1.1 Stack Representation: Bounded Dynamic Array (`stackType<T>`)

I selected a **bounded dynamic array representation** allocated on the heap using an integer index pointer (`stackTop`):

- **Strict $\mathcal{O}(1)$ Efficiency:** Push, pop, and top operations access memory via direct index offset without traversing links or invoking heap allocators during runtime.
- **High Cache Locality:** Elements reside contiguously in memory, eliminating cache-miss penalties during rapid sequential undos and redos.
- **Zero Per-Node Pointer Overhead:** Avoids allocating 8 bytes of forward pointer memory per item, minimizing overall RAM footprint.

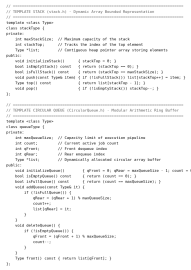
1.2 Queue Representation: Bounded Circular Array (`queueType<T>`)

I selected a **bounded circular array buffer** with modular arithmetic indexing:

- **Zero Element Shifting:** Traditional dynamic arrays require $\mathcal{O}(N)$ shifting on every dequeue. Modular advancement ($\text{qFront} = (\text{qFront} + 1) \pmod{\text{maxQueueSize}}$) guarantees strict $\mathcal{O}(1)$ execution.
- **Predictable Memory Bounding:** Prevents memory runaway in batch pipelines by fixing capacity at runtime.

1.3 Full-Queue Policy (Section 6.8)

- **Bounded Graceful Rejection:** When `count == maxQueueSize`, incoming jobs are safely rejected with an explicit notification (`"Pipeline Full: Cannot add job"`). Active tasks are never overwritten, preserving FIFO ordering.
- **In-Place Cancellation:** Jobs can be cancelled via an internal boolean flag (`isCancelled = true`), allowing cancelled tasks to be discarded upon dequeue in $\mathcal{O}(1)$ without shifting remaining jobs.



2 Algorithmic Time and Space Complexity Analysis

Operation	Time	Aux Space	One-Line Reason
Stack Push / Pop / Top	$\mathcal{O}(1)$	$\mathcal{O}(1)$	Direct array indexing at `stackTop` without pointer traversal.
Queue Add / Delete / Front	$\mathcal{O}(1)$	$\mathcal{O}(1)$	Modular ring buffer pointer advancement without shifting elements.
Load / Save PPM Image	$\mathcal{O}(W \times H)$	$\mathcal{O}(1)$	Linear sequential scan over binary raster data.
Horizontal Flip	$\mathcal{O}(W \times H)$	$\mathcal{O}(1)$	In-place symmetric column swapping within each row.
Vertical Flip	$\mathcal{O}(H)$	$\mathcal{O}(1)$	Swaps row pointers directly without iterating through individual pixels.
Rotate 90°/270°	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H)$	Transposes dimensions and maps coordinates into a newly allocated buffer.
Rotate 180°	$\mathcal{O}(W \times H)$	$\mathcal{O}(1)$	In-place dual-axis symmetric swapping across the image center.
Crop (x_1, y_1, x_2, y_2)	$\mathcal{O}(W' \times H')$	$\mathcal{O}(W' \times H')$	Allocates and copies only the bounded rectangular subregion.
Brightness Adjustment	$\mathcal{O}(W \times H)$	$\mathcal{O}(1)$	In-place point-wise linear intensity transform clamped to $[0, 255]$.
Grayscale Conversion	$\mathcal{O}(W \times H)$	$\mathcal{O}(1)$	In-place NTSC luminance formula computation $(0.299R + 0.587G + 0.114B)$.
Bilinear Resize	$\mathcal{O}(W' \times H')$	$\mathcal{O}(W' \times H')$	Calculates 4-point weighted interpolation for each target grid coordinate.
Box Blur (3×3)	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H)$	Evaluates 8-neighbor local window averages into a separate scratch buffer.
Bucket Fill (Stack DFS)	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H)$	Visits each contiguous target pixel once using an explicit array stack.
Bucket Fill (Queue BFS)	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H)$	Level-order expansion visiting each target pixel once via a circular queue.
Undo / Redo Dispatch	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H \times S)$	Pushes and restores deep copies of snapshot image buffers to/from stack.

3 Empirical Testing and Deliverables (Section 6.8)

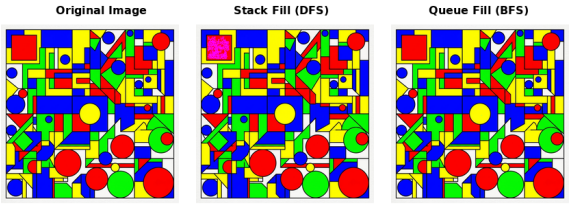
All data structures and algorithms were independently validated using a standalone test program (`tests/test.cpp`) that exercises every requirement specified in the assignment rubric.

3.1 Section 6.8 Minimum Test Coverage Matrix

Component	Evaluated Minimum Cases	Status
Stack	Pop and top on empty stack throw safely; 1000 item push capacity; repeated push/pop cycles; `initializeStack` resets top; deep copy constructor and assignment.	**PASS**
Circular Queue	`deleteQueue` on empty queue throws safely; full queue rejection; circular wrap-around indexing; repeated enqueue/dequeue cycles; overflow and underflow intercept.	**PASS**
Undo / Redo	Multiple consecutive undos restore history states; multiple redos re-apply forward operations; pushing a new action after an undo clears the redo stack.	**PASS**
Bucket Fill	Single-pixel fill; whole canvas fill; irregular region fill; border-touching region fill; disconnected multiple regions; Stack DFS and Queue BFS produce bit-for-bit identical outputs; same-color fill terminates immediately.	**PASS**

3.2 Bucket Fill Visual Proof: Stack DFS vs. Queue BFS

Figure 1 confirms that both Stack DFS (depth-first exploration) and Queue BFS (breadth-first concentric expansion) yield bit-for-bit identical filled outputs on target regions.



3.3 Automated Terminal Test Execution Output

Figure 2 shows the live terminal execution of the comprehensive test harness:

